



Department of Computer Science and Engineering
Premier University

CSE 454 : Compiler Construction Laboratory

Title: Design and Implementation of a Lexical Analyzer to Count
Identifiers in a C Program
Lab: 05

Submitted by:

Name	Bibi Hazaratun Nesa
ID	0222210005101101
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akhter
Assistant Professor
Department of CSE
Premier University
Chittagong

Remarks

Lab Report

Experiment Title:

Design and Implementation of a Lexical Analyzer to Count Identifiers in a C Program

1. Objective

The objective of this experiment is to design and implement a simple lexical analyzer in C that reads a C source file and identifies all unique identifiers present in the program. The program should list the identifiers and count the total number of unique identifiers.

2. Theory

A compiler works in different phases. The first phase is called **Lexical Analysis**. In this phase, the source code is scanned character by character and converted into meaningful tokens such as keywords, identifiers, operators, constants, and symbols.

An **identifier** is the name used to identify variables, functions, arrays, or other user-defined entities in a program.

Rules of a Valid Identifier in C:

1. It must start with a letter (a–z, A–Z) or an underscore (_).
2. It can contain letters, digits (0–9), and underscores.
3. It must not be a C keyword.
4. Identifiers are case-sensitive.

In this experiment, a lexical analyzer is implemented to:

- Read a C source file.
- Ignore preprocessor directives (e.g., `#include`).
- Ignore string literals.
- Ignore C keywords.
- Remove duplicate identifiers.
- Display the list of unique identifiers.
- Display the total count of identifiers.

3. Algorithm

1. Start the program.
2. Ask the user to enter the file name.
3. Open the file in read mode.
4. Read the file character by character.
5. If a preprocessor directive (`#`) is found, ignore the entire line.
6. If a string literal (`" "`) is found, ignore everything inside the quotes.
7. Form words using alphanumeric characters and underscores.
8. Check if the word:
 - Starts with a letter or underscore.
 - Is not a keyword.
9. If valid, check whether it already exists in the identifier list.
10. If it is unique, store it.
11. Continue until end of file.
12. Print all identifiers.
13. Print total count.
14. Stop.

5. Source Code

```
Start here x *lab_07.c x identifier_5.c x
1  #include <stdio.h>
2  #include <ctype.h>
3  #include <string.h>
4
5  #define MAX_ID 1000
6  #define MAX_LEN 100
7
8  char *keywords[] = {
9      "int", "float", "double", "char", "void", "if", "else",
10     "while", "for", "do", "return", "break", "continue",
11     "switch", "case", "default", "struct", "union",
12     "typedef", "const", "static", "sizeof", "long",
13     "short", "unsigned", "signed", "goto", "enum"
14 };
15
16 int isKeyword(char *word) {
17     int n = sizeof(keywords)/sizeof(keywords[0]);
18     for(int i = 0; i < n; i++) {
19         if(strcmp(word, keywords[i]) == 0)
20             return 1;
21     }
22     return 0;
23 }
24
25 int isIdentifier(char *word) {
26     if(!(isalpha(word[0]) || word[0] == '_'))
27         return 0;
28
29     for(int i = 1; word[i] != '\0'; i++) {
30         if(!(isalnum(word[i]) || word[i] == '_'))
31             return 0;
32     }
```

C:\Users\iQ Tech\OneDrive\Documents\identifier_5.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 20, Col 22, Po

```
Start here x *lab_07.c x identifier_5.c x
36     return 0;
37
38     return 1;
39 }
40
41 int main() {
42
43     char filename[100];
44     FILE *fp;
45     char ch, buffer[MAX_LEN];
46     char identifiers[MAX_ID][MAX_LEN];
47
48     int i = 0, count = 0, exists;
49
50     printf("Enter file name: ");
51     scanf("%s", filename);
52
53     fp = fopen(filename, "r");
54
55     if(fp == NULL) {
56         printf("Error opening file!\n");
57         return 1;
58     }
59
60     while((ch = fgetc(fp)) != EOF) {
61
62         // Ignore preprocessor line
63         if(ch == '#') {
64             while((ch = fgetc(fp)) != '\n' && ch != EOF);
65             continue;
66         }
67     }
```

```
71         continue;
72     }
73
74     if(isalnum(ch) || ch == '_' ) {
75         buffer[i++] = ch;
76     }
77     else {
78         if(i > 0) {
79             buffer[i] = '\0';
80             i = 0;
81
82             if(isIdentifier(buffer)) {
83
84                 exists = 0;
85                 for(int j = 0; j < count; j++) {
86                     if(strcmp(identifiers[j], buffer) == 0) {
87                         exists = 1;
88                         break;
89                     }
90                 }
91
92                 if(!exists) {
93                     strcpy(identifiers[count], buffer);
94                     count++;
95                 }
96             }
97         }
98     }
99 }
100
101 fclose(fp);
102
```

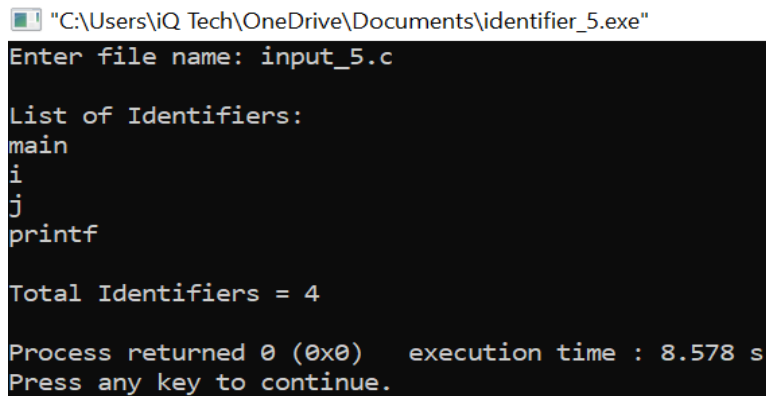
```
94         count++;
95     }
96 }
97
98 }
99
100
101 fclose(fp);
102
103 printf("\nList of Identifiers:\n");
104 for(int j = 0; j < count; j++) {
105     printf("%s\n", identifiers[j]);
106 }
107
108 printf("\nTotal Identifiers = %d\n", count);
109
110 return 0;
111 }
112
```

C:\Users\iQ Tech\OneDrive\Documents\identifier_5.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 97, Col 14, Pos 2318 Ove

5. Sample Input (input.c)

```
1 #include<stdio.h>
2 void main()
3 {
4     int i,j;
5     printf("it's a C program");
6 }
7
```

6. Sample Output



```
"C:\Users\iQ Tech\OneDrive\Documents\identifier_5.exe"
Enter file name: input_5.c

List of Identifiers:
main
i
j
printf

Total Identifiers = 4

Process returned 0 (0x0)   execution time : 8.578 s
Press any key to continue.
```

7. Result

The program was successfully implemented and tested.

It correctly identified all unique identifiers from the given C source file, ignored keywords, string literals, and preprocessor directives, and displayed the total number of unique identifiers.

8. Conclusion

This experiment demonstrates the basic concept of lexical analysis, which is the first phase of a compiler. By implementing this program, we understood how tokens are formed and how identifiers can be separated from keywords and other elements of a C program.

The experiment was completed successfully.



Department of Computer Science and Engineering
Premier University

CSE 454 : Compiler Construction Laboratory

Title: write a program to detect and remove left recursion from a given context free grammar.

Submitted by:

Name	Bibi Hazaratun Nesa
ID	0222210005101101
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akhter
Assistant Professor
Department of CSE
Premier University
Chittagong

Remarks

Experiment No : 06

Experiment Name : write a program to detect and remove left recursion from a given context free grammar.

Objective: The objective of this program is to detect and remove left recursion from a given Context Free Grammar (CFG) using the C programming language. Left recursion occurs when a non-terminal appears as the first symbol in its own production rule, such as $A \rightarrow A\alpha \mid \beta$, which creates problems for top-down parsing techniques.

Algorithm:

1. Start the program and create the source file CC_Lab_6.c.
2. Include the required header files `stdio.h` and `string.h` for input/output operations and string handling.
3. Define constants to represent the maximum number of non-terminals and the maximum length of production rules.
4. Declare the necessary variables and arrays to store non-terminals, production rules, and temporary strings for processing.
5. Ask the user to enter the number of non-terminals in the context free grammar.
6. Using a loop, take input for each non-terminal symbol and its production rule.
7. Split the production rules using the '`|`' (vertical bar) symbol to separate multiple productions.
8. Check each production to determine whether the first symbol is the same as the non terminal.
9. If the production starts with the same non-terminal, identify it as left recursive and store the remaining part in the alpha list.
10. If the production does not start with the non-terminal, store it in the beta list.
11. After checking all productions, determine whether alpha productions exist, which indicates the presence of left recursion.
12. If left recursion is found, apply the transformation rule: $A \rightarrow \beta A' \mid A' \rightarrow \alpha A' \mid \epsilon$
13. Display the modified grammar without left recursion.
14. Save the program in `main6.c`, compile it, run the program, and display the output.
15. End the program.

Source Code:

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 10
5  #define SIZE 20
6
7  int main() {
8      int n, i, j, k;
9      char nonterminals[MAX];
10     char productions[MAX][SIZE];
11     char temp[SIZE];
12
13     printf("Enter number of non-terminals: ");
14     scanf("%d", &n);
15
16
17     for(i = 0; i < n; i++) {
18         printf("Enter non-terminal %d: ", i+1);
19         scanf("%c", &nonterminals[i]);
20
21         printf("Enter production for %c (use | for alternatives): ", nonterminals[i]);
22         scanf("%s", productions[i]);
23     }
24
25     printf("\nOriginal Grammar:\n");
26     for(i = 0; i < n; i++)
27         printf("%c -> %s\n", nonterminals[i], productions[i]);
28
29     printf("\nRemoving Left Recursion...\n");
30
31     for(i = 0; i < n; i++) {
32         char alpha[SIZE][SIZE];
33
34         char *token = strtok(productions[i], "|");
35         while(token != NULL) {
36             if(token[0] == nonterminals[i]) {
37                 strcpy(alpha[alphaCount++], token + 1);
38             } else {
39                 strcpy(beta[betaCount++], token);
40             }
41             token = strtok(NULL, "|");
42         }
43
44         if(alphaCount > 0) {
45             printf("%c -> ", nonterminals[i]);
46             for(j = 0; j < betaCount; j++) {
47                 printf("%s%c", beta[j], nonterminals[i]);
48                 if(j != betaCount-1) printf(" | ");
49             }
50             printf("\n");
51
52             printf("%c' -> ", nonterminals[i]);
53             for(j = 0; j < alphaCount; j++) {
54                 printf("%s%c", alpha[j], nonterminals[i]);
55                 if(j != alphaCount-1) printf(" | ");
56             }
57             printf(" | %s\n", nonterminals[i]);
58         } else {
59             printf("%c -> %s\n", nonterminals[i], productions[i]);
60         }
61     }
62
63     return 0;
64 }

```

❖ The program in main6.c is designed to detect and remove left recursion from a given context-free grammar (CFG). It first takes input from the user, including the number of non-terminals, each non-terminal symbol, and its production rules, where alternatives are separated by the | symbol.

❖ For each non-terminal, the program examines all its productions to determine whether left recursion exists, i.e., whether a production begins with the same non-terminal. The productions are separated into alpha (left-recursive) and beta (non-recursive) parts. If left recursion is detected, the program applies the standard transformation rule to eliminate it:

❖ $A \rightarrow \beta A'$

❖ $A' \rightarrow \alpha A' \mid \epsilon$

❖ Finally, the program prints the new grammar without left recursion. If no left recursion is present, it simply prints the original grammar.

Output:

```
"C:\Users\iQ Tech\OneDrive\Documents\CC_Lab_6.exe"
Enter number of non-terminals: 3
Enter non-terminal 1: S
Enter production for S (use | for alternatives): Sa|bS
Enter non-terminal 2: Ab|d
Enter production for A (use | for alternatives): Enter non-terminal 3: Be|fd
Enter production for B (use | for alternatives):
Original Grammar:
S -> Sa|bS
A -> b|d
B -> e|fd

Removing Left Recursion...
S -> bSS'
S' -> aS' |  $\epsilon$ 
A -> b
B -> e

Process returned 0 (0x0)   execution time : 75.947 s
Press any key to continue.
```

❖ The program first asks the user to enter the number of non-terminals and their corresponding production rules. In this case, three non-terminals S, A, and B were entered with productions using the | symbol to separate alternatives.

❖ Next, it detects left recursion in the grammar. For each non-terminal, the left-recursive and non-recursive parts are separated, and the grammar is transformed using the standard left recursion removal rule. New non-terminals (S', A', B') are introduced, and ϵ is used to represent the empty string, allowing the recursion to terminate.

❖ Finally, the program prints the modified grammar without left recursion, showing that all rules have been successfully rewritten.

Discussion:

In this experiment, a program was implemented in C to detect and remove left recursion from a given Context Free Grammar (CFG). Left recursion occurs when a non-terminal appears as the first symbol in one of its own productions, such as $A \rightarrow A\alpha \mid \beta$. This type of grammar creates difficulties for top-down parsing techniques because it may lead to infinite recursion during parsing. In the program, the grammar productions are analyzed and separated into two parts: alpha (recursive productions) and beta (non-recursive productions). The grammar is then rewritten using a new non-terminal symbol to remove the left recursion. The transformed grammar follows the standard form $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$, which eliminates the left recursive structure. As a result, the modified grammar becomes suitable for top-down parsing methods such as recursive descent parsing. This experiment demonstrates how grammar transformation techniques can be used in compiler design to make grammars easier and more efficient for parsing.



Department of Computer Science and Engineering
Premier University

CSE 454 : Compiler Construction Laboratory

Title: Write a Program to apply Algorithm for Left Factoring a
Context-Free Grammar (CFG)
Lab: 07

Submitted by:

Name	Bibi Hazaratun Nesa
ID	0222210005101101
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akhter
Assistant Professor
Department of CSE
Premier University
Chittagong

Remarks

Lab Report: 07

Title:

Write a Program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG)

Objective :

To transform a given Context-Free Grammar (CFG) by removing common prefixes in productions, making it suitable for top-down parsing (e.g., recursive descent parsing).

Theory:

Left factoring is a grammar transformation technique used in compiler design to eliminate ambiguity and make a grammar suitable for predictive parsing.

If a non-terminal has productions like:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n$$

Where **α is a common prefix**, then it can be rewritten as:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

This helps the parser to make decisions without backtracking.

Algorithm:

1. Start
2. For each non-terminal $A \in V$, check if it has productions of the form:
 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$
3. If common prefix α exists:
 - o Replace with:
 - $A \rightarrow \alpha A'$
 - $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$
 - o Include ϵ if necessary
4. Repeat for all non-terminals
5. End

Program (C Code):

```
Start here x *lab_07.c x
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char input[100], lhs;
6      char productions[10][50];
7      int n = 0;
8
9      printf("Enter CFG : ");
10     scanf("%s", input);
11
12     lhs = input[0];
13
14
15     char *rhs = strstr(input, "->") + 2;
16
17
18     char *token = strtok(rhs, "|");
19     while (token != NULL) {
20         strcpy(productions[n++], token);
21         token = strtok(NULL, "|");
22     }
23
24
```

```
Start here x *lab_07.c x
27
28     for (int i = 1; i < n; i++) {
29         int j = 0;
30         while (prefix[j] && productions[i][j] && prefix[j] == productions[i][j]) {
31             j++;
32         }
33         prefix[j] = '\0';
34     }
35
36
37     if (strlen(prefix) == 0) {
38         printf("\nNo Left Factoring needed.\n");
39         return 0;
40     }
41
42     printf("\nAfter Left Factoring:\n");
43     printf("%c -> %s%c\n", lhs, prefix, lhs);
44     printf("%c' -> ", lhs);
45
46     for (int i = 0; i < n; i++) {
47         if (strlen(productions[i]) == strlen(prefix))
48             printf("&");
49         else
50             printf("%s", productions[i] + strlen(prefix));
51
52         if (i != n - 1)
53             printf(" | ");
54     }
55
56     printf("\n");
57
58     return 0;
59
60
61
```

Sample Output:

"C:\Users\iQ Tech\OneDrive\Documents\lab_07.exe"

Enter CFG : A->abC|abD|aE

After Left Factoring:

A -> aA'

A' -> bC | bD | E

Process returned 0 (0x0) execution time : 41.341 s

Press any key to continue.

Result:

The program successfully applies the left factoring algorithm to the given CFG.

Conclusion:

The experiment shows how left factoring removes common prefixes from productions, making the grammar suitable for efficient top-down parsing.